

Case Study 1

Name	Question Number	Description
Mansi Jayeshbhai Sutreja	Q1, Q2	How many records have a date that is expressed without using alphabets? How many records have a word starting with the letter "w"?
Siddhi Pravinbhai Patel	Q3, Q4	How many records make a word that starts with an alphabet and is not a URL? How many tweets contain one of these emojis :), :D, ;), :P?
Utsav Harshadbhai Khamar	Q5, Q6	How many records contain a decimal number? What is the total number of IP addresses across all the records?
Kauthara Oyinkansola Yakubu	Q7, Q8	How many records have a new line? What is the total number of hashtags across all these tweets?
Abdullah Iftaqar Mohammed	Q9, Q10	What is the code to substitute all non-alphanumeric characters with a new line? What is the total number of URLs across all tweets?
Shahnaz Palakunnil Moosa	B1	Use Porter stemmer to run stemming. Count the number of unique words/tokens before and after stemming.
Clifford Addison	B2	Perform lemmatization using NLTK lemmatizer. Count the number of unique words/tokens before and after lemmatization.
Obianuju Nonyerem Anuma	B3	Compare the change in word frequencies from stemming and lemmatization. Which are the top 10 words after stemming/lemmatization?
Saurav Risal	B4	What is the change in word frequencies if normalization is done after stop word removal?

Question 1

```
In [ ]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
In [ ]: cd drive/MyDrive/Data/

/content/drive/MyDrive/Data
```

```
In [ ]: import pandas as pd

file_path = "/content/drive/MyDrive/Data/Data.csv"
df = pd.read_csv(file_path)
```

```
In [ ]: print("\nDataset Info:")
        print(df.info())
```

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7339 entries, 0 to 7338
Data columns (total 1 columns):
 #   Column  Non-Null Count  Dtype
---  -
 0   Data    7339 non-null     object
dtypes: object(1)
memory usage: 57.5+ KB
None
```

```
In [ ]: print("\nSample Data:")
        print(df.head())
```

```
Sample Data:

                                Data
0  Watch or listen live weekdays at 8:30am MT at ...
1  Watch or listen live weekdays at 8:30am MT at ...
2                Chubby And Hot, Always Stir The Pot!
3                Chubby And Hot, Always Stir The Pot!
4  Journalist, publisher of Rebel News – telling ...
```

```
In [ ]: data_pattern = r'\b(?:\d{1,2}[\./\-\.\.]){2}\d{2,4}\b|\b\d{4}[\./\-\.\.](?:\d{1,2}[\./\-\.\.]){2}\d{2,4}\b'
```

```
In [ ]: import re
df['Has_Number'] = df['Data'].apply(lambda x: bool(re.search(data_pattern, x)))
```

```
In [ ]: data_count = df['Has_Number'].sum()
```

```
In [ ]: print(f"Number of records containing any number (no alphabets): {data_count}")

Number of records containing any number (no alphabets): 0
```

Question 2

```
In [ ]: w_word_pattern = r'\b\w*'
```

```
In [ ]: df['Has_W_Word'] = df['Data'].apply(lambda x: bool(re.search(w_word_pa
```

```
In [ ]: w_word_count = df['Has_W_Word'].sum()
```

```
In [ ]: print(f"Number of records containing a word starting with 'w': {w_word
```

Number of records containing a word starting with 'w': 3591

Chat GPT prompt :

- give me all possible ways to get number of date present in any data without any alphabets
- want the same to find number of records having word starting with 'w' for both upper case and lower case

Question 3

```
In [ ]: import pandas as pd
import re
```

```
In [ ]: # STEP 1: Load text file without headers (just plain text)
df = pd.read_csv('Downloads/corpus.csv', header=None, names=['text'])
```

```
In [ ]: # STEP 2: Define your cleaning function
def clean_text(text):
    text = re.sub(r'^\x00-\x7F+', ' ', str(text)) # remove non-ASCII
    text = re.sub(r'\s+', ' ', text) # normalize white
    return text.strip()
```

```
In [ ]: # STEP 3: Apply cleaning
df['clean_text'] = df['text'].apply(clean_text)
```

```
In [ ]: # STEP 4: Drop duplicates based on cleaned text
df_cleaned = df.drop_duplicates(subset='clean_text').reset_index(drop=
```

```
In [ ]: # STEP 5: save result
print(df_cleaned.head())
```

```

                                text \
0                                Data
1 Watch or listen live weekdays at 8:30am MT at ...
2           Chubby And Hot, Always Stir The Pot!
3 Journalist, publisher of Rebel News – telling ...
4 Edmonton City Councillor for #WardMétis. #YEG ...

```

```

                                clean_text
0                                Data
1 Watch or listen live weekdays at 8:30am MT at ...
2           Chubby And Hot, Always Stir The Pot!
3 Journalist, publisher of Rebel News telling th...
4 Edmonton City Councillor for #WardM tis. #YEG ...

```

```

In [ ]: def is_valid_word(word):
        # Check if word starts with an alphabet and is not a URL
        return re.match(r'^[a-zA-Z]', word) and not re.match(r'^(http|http

```

```

In [ ]: def has_valid_word(text):
        words = text.split()
        return any(is_valid_word(word) for word in words)

```

```

In [ ]: # Apply to each row to check if it contains at least one valid word
df_cleaned['has_valid_word'] = df_cleaned['clean_text'].apply(has_vali

```

```

In [ ]: # Count how many rows have at least one valid word
count_valid_records = df_cleaned['has_valid_word'].sum()

```

```

In [ ]: print(f"\033[1mNumber of records with at least one valid word (starts
Number of records with at least one valid word (starts with an alphabet
and not a URL): 1956

```

Question 4

```

In [ ]: emojis = [":)", ":D", ";)", ":P"]

```

```

In [ ]: def contains_emoji(text):
        return any(e in text for e in emojis)

```

```

In [ ]: # Counts and prints the number of tweets that contain at least one emo

mask = df['text'].apply(contains_emoji)
count = mask.sum()
print(f"\033[1mNumber of tweets containing at least one of the emojis:

```

Number of tweets containing at least one of the emojis: 18

ChatGPT: first and last prompts*

Determine the total count of tokens that initiate with an alphabetical character and exclude any entries resembling web addresses

Question 5

```
In [ ]: import pandas as pd
import re
```

```
In [ ]: corpus = pd.read_csv(r"C:\Users\utsav\Downloads\Data.csv")
```

```
In [ ]: import pandas as pd
import re

# Regular expression pattern to match decimal numbers
decimal_pattern = r'\d+\.\d+'

# Regular expression pattern to match US-style phone numbers
phone_pattern = r'"""
(?:((?:\+?1[\s.-])?)?      # Optional country code: +1, 1, etc.
(?:\((?d{3})\)?[\s.-])?    # Area code: (123), 123, etc.
\d{3}[\s.-]?d{4})          # Final 7 digits of the phone number
"""

# Compile the phone pattern with re.VERBOSE to allow comments and form
compiled_phone_pattern = re.compile(phone_pattern, re.VERBOSE)

# Function to extract all decimal numbers from text
def extract_decimals(text):
    return re.findall(decimal_pattern, text)

# Function to extract all phone numbers from text
def extract_phones(text):
    return compiled_phone_pattern.findall(text)

# Function to remove decimal numbers that appear inside phone numbers
def filter_decimals(text, decimals, phones):
    cleaned = []
    for dec in decimals:
        # Keep only those decimals that are NOT part of any phone number
        if not any(dec in phone for phone in phones):
            cleaned.append(dec)
    return cleaned

# Extract phone numbers into a new column
corpus['phone_numbers'] = corpus['Data'].apply(extract_phones)

# Extract all raw decimal numbers into a new column
```

```

corpus['raw_decimals'] = corpus['Data'].apply(extract_decimals)

# Remove decimals that are part of phone numbers and store cleaned list
corpus['valid_decimals'] = corpus.apply(
    lambda row: filter_decimals(row['Data'], row['raw_decimals'], row['axis=1']
)

# Count how many valid decimal numbers exist in each row
corpus['decimal_count'] = corpus['valid_decimals'].apply(len)

# Filter the rows where at least one valid decimal was found
corpus_filtered = corpus[corpus['decimal_count'] > 0][['Data', 'valid_

# Print the filtered rows in a readable format
for idx, row in corpus_filtered.iterrows():
    print(f"Row index {idx} contains decimals: {row['valid_decimals']}")
    print(f"Line: {row['Data']}\n")
    print("-----")

```

Row index 2388 contains decimals: ['99.1']
 Line: Don Hladiuk is Calgary's eye on astronomy & space science news. On the CBC EyeOpener 1010AM or 99.1FM on the 1st or 2nd Monday of the month at 7:36 AM MT.

 Row index 2718 contains decimals: ['99.1']
 Line: Don Hladiuk is Calgary's eye on astronomy & space science news. On the CBC EyeOpener 1010AM or 99.1FM on the 1st or 2nd Monday of the month at 7:36 AM MT.

 Row index 3208 contains decimals: ['106.1']
 Line: AIR 106.1 FM + DiscoverAirdrie Local News.
 Want to win a truck? Our Ram Everyday Adventure Contest is BACK!

@RadioClaireMC

 Row index 3663 contains decimals: ['26.4']
 Line: hier schreiben die Hühner aus dem Stall (*25./26.4. 2019) .
 Körner und Würmer immer willkommen

 Row index 3861 contains decimals: ['106.1']
 Line: AIR 106.1 FM + DiscoverAirdrie Local News.
 Want to win a truck? Our Ram Everyday Adventure Contest is BACK!

@RadioClaireMC

 Row index 4316 contains decimals: ['26.4']
 Line: hier schreiben die Hühner aus dem Stall (*25./26.4. 2019) .

Körner und Würmer immer willkommen

Row index 4568 contains decimals: ['3.905', '3.2']
Line: #openaccess journal published by@MDPIOpenAccess, covering aspects of #landscience. IF 2021: 3.905; CiteScore 2021: 3.2.
#LULC #landscape #soil #urban #climate

Row index 4626 contains decimals: ['2.0']
Line: Hoping hardcore twitter 2.0 lets me express honest opinions about Danielle Smith's supporters.

"Let them see what is on the end of that long newspaper spoon."

Row index 4943 contains decimals: ['106.1']
Line: AIR 106.1 FM + DiscoverAirdrie Local News.
Want to win a truck? Our Ram Everyday Adventure Contest is BACK!

@RadioClaireMC

Row index 5113 contains decimals: ['100.7']
Line: Playing Southern Saskatchewan's Best Country! Country 100.7 FM!

Row index 5231 contains decimals: ['106.1']
Line: AIR 106.1 FM + DiscoverAirdrie Local News.
Want to win a truck? Our Ram Everyday Adventure Contest is BACK!

@RadioClaireMC

Row index 5244 contains decimals: ['96.5', '96.5']
Line: 96.5 CKFM – All Hit Country!
 96.5 FM
 ckfm.ca
Your source for everything happening in Mountain View County!

Row index 5263 contains decimals: ['96.5', '96.5']
Line: 96.5 CKFM – All Hit Country!
 96.5 FM
 ckfm.ca
Your source for everything happening in Mountain View County!

Row index 5445 contains decimals: ['106.1']
Line: AIR 106.1 FM + DiscoverAirdrie Local News.
Want to win a truck? Our Ram Everyday Adventure Contest is BACK!

@RadioClaireMC

Row index 5887 contains decimals: ['3.905', '3.2']
Line: #openaccess journal published by@MDPIOpenAccess, covering aspects of #landscience. IF 2021: 3.905; CiteScore 2021: 3.2.
#LULC #landscape #soil #urban #climate

Row index 5945 contains decimals: ['2.0']
Line: Hoping hardcore twitter 2.0 lets me express honest opinions about Danielle Smith's supporters.

"Let them see what is on the end of that long newspaper spoon."

Row index 6262 contains decimals: ['106.1']
Line: AIR 106.1 FM + DiscoverAirdrie Local News.
Want to win a truck? Our Ram Everyday Adventure Contest is BACK!

@RadioClaireMC

Row index 6432 contains decimals: ['100.7']
Line: Playing Southern Saskatchewan's Best Country! Country 100.7 FM!

Row index 6550 contains decimals: ['106.1']
Line: AIR 106.1 FM + DiscoverAirdrie Local News.
Want to win a truck? Our Ram Everyday Adventure Contest is BACK!

@RadioClaireMC

Row index 6563 contains decimals: ['96.5', '96.5']
Line: 96.5 CKFM – All Hit Country!
 96.5 FM
 ckfm.ca
Your source for everything happening in Mountain View County!

Row index 6582 contains decimals: ['96.5', '96.5']
Line: 96.5 CKFM – All Hit Country!
 96.5 FM
 ckfm.ca
Your source for everything happening in Mountain View County!

Row index 6764 contains decimals: ['106.1']
Line: AIR 106.1 FM + DiscoverAirdrie Local News.
Want to win a truck? Our Ram Everyday Adventure Contest is BACK!

@RadioClaireMC

```
In [ ]: print(f"Total rows with standalone decimals: {len(corpus_filtered)}")
```

Total rows with standalone decimals: 22

ChatGPT Prompts

First: how would you define a decimal number, would you consider integers or strictly those with a decimal point?

Last: suggest me a way to properly display the list of such rows.

Question 6

```
In [ ]: # Regular expression to match IPv4 addresses (e.g., 192.168.1.1)
ip_pattern = r'''
(?:
    (?:(?:25[0-5]      # 250-255
        | 2[0-4][0-9]  # 200-249
        | 1[0-9]{2}    # 100-199
        | [1-9]?[0-9]) # 0-99
    \.
){3}
(?:25[0-5]|2[0-4][0-9]|1[0-9]{2}|[1-9]?[0-9])
'''

# Compile with VERBOSE to support comments and formatting
compiled_ip_pattern = re.compile(ip_pattern, re.VERBOSE)

# Function to extract strictly valid IPv4 addresses
def extract_strict_ips(text):
    return compiled_ip_pattern.findall(text)

# Apply to DataFrame
corpus['ip_addresses'] = corpus['Data'].apply(extract_strict_ips)
corpus['ip_count'] = corpus['ip_addresses'].apply(len)
```

```
In [ ]: #total IP count
total_ips = df['ip_count'].sum()
print(f"Total valid IPv4 addresses: {total_ips}\n")

#Show rows that contain IPs
df_with_ips = df[df['ip_count'] > 0][['Data', 'ip_addresses']]
```

```

for idx, row in df_with_ips.iterrows():
    print(f"Row index {idx} contains IPs: {row['ip_addresses']}")
    print(f"Line: {row['Data']}\n")

#Get a flat list of all IPs across all rows
all_ips = [ip for sublist in df['ip_addresses'] for ip in sublist]
print("List of all valid IP addresses:")
print(all_ips)

```

Total valid IPv4 addresses: 0

List of all valid IP addresses:
[]

ChatGPT Prompts

First: what is the proper format of ipv4 addresses?

Last: Multiple lines in regular expression

Question 7

Question	Answer
7. How many records have a new line?	Number of records with new lines: 1211
8. What is the total number of hashtags across all these tweets?	Total number of hashtags: 2924

[]

```

In [ ]: # importing libraries
import pandas as pd
import re

```

```

In [ ]: # Uploading file
corpus = pd.read_csv('/content/Data.csv')

```

```

In [ ]: #checking the first 10 row
corpus.head(10)

```

Out []:

Data

	Data
0	Watch or listen live weekdays at 8:30am MT at ...
1	Watch or listen live weekdays at 8:30am MT at ...
2	Chubby And Hot, Always Stir The Pot!
3	Chubby And Hot, Always Stir The Pot!
4	Journalist, publisher of Rebel News — telling ...
5	Edmonton City Councillor for #WardMétis. #YEG ...
6	Albertan. Mom to children & dogs. Wife. Friend...
7	Albertan. Mom to children & dogs. Wife. Friend...
8	Albertan. Mom to children & dogs. Wife. Friend...
9	Albertan. Mom to children & dogs. Wife. Friend...

```
In [ ]: # Count Records with New Lines
new_line_count = corpus['Data'].str.contains('\n').sum()
```

```
In [ ]: print(f"Number of records with new lines: {new_line_count}")
```

Number of records with new lines: 1211

Question 8

```
In [ ]: # Function counting the hashtags in a tweet
def count_hashtags(tweet):
    return len(re.findall(r'#\w+', tweet))
```

```
In [ ]: # Apply the function to the dataset column and sum the results
total_hashtags = corpus['Data'].apply(count_hashtags).sum()
```

```
In [ ]: print(f"Total number of hashtags: {total_hashtags}")
```

Total number of hashtags: 2924

Blackbox Prompt Used

Show me how to count record with new line

Show me how to count hashtag in a tweet using RegEx

Question 9

```
In [ ]: import pandas as pd
import re
```

```
In [ ]: corpus = pd.read_csv("C:\\Users\\500236394\\Downloads\\Data.csv")
```

ChatGPT prompt

9. What is the code to substitute all non-alphanumeric characters

with a new line?

10. What is the total number of URLs across all tweets?

```
In [ ]: #Iterating over 3 rows of data and replace non alphanumeric with \n
for index, row in corpus.head(3).iterrows(): #only working on 3 rows
    text = row['Data']
    modified_text = re.sub(r'^a-zA-Z0-9', '\n', text) # substitute a
    print(modified_text)
```

Watch
or
listen
live
weekdays
at
8
30am
MT
at
ryanjespersen
com

Subscribe
via
YouTube
or
your
favourite
podcast
app

RealTalkRJ
Watch
or
listen
live
weekdays

at
8
30am
MT
at
ryanjespersen
com

Subscribe
via
YouTube
or
your
favourite
podcast
app

RealTalkRJ
Chubby
And
Hot

Always
Stir
The
Pot

Question 10

```
In [ ]: url_pattern = r'https?:\/\/[^\s]+' #url regex pattern
        no_of_urls=0 #counter for urls
        for index, row in corpus.iterrows():
            text = row['Data']
            no_of_urls += len(re.findall(url_pattern, text)) # adds len of res
        print(no_of_urls)
```

4

Question B.1.

LLM used: ChatGPT Prompts first: Write a python code in ipynb format for Perform stemming and lemmatization (40%), Use porter stemmer to run stemming. Count the number of unique words/tokens before and after stemming Prompt last: explain why nltk punkt library used

```
In [ ]: # Import necessary libraries
```

```
import pandas as pd

from nltk.tokenize import word_tokenize

from nltk.stem import PorterStemmer

from nltk.corpus import stopwords

import string

import nltk
import re

# Download required NLTK data for handling punctations
nltk.download('punkt')

# Import the corpus
corpus = pd.read_csv('C:\\PG AI and DS\\Semester 3\\NLP AISC2009\\Data

# Check the first 5 documents
corpus.head()

# Combine all text data into a single string (adjust the column name a
text_data = " ".join(corpus.astype(str).apply(" ".join, axis=1))

# Tokenize the text
tokens = word_tokenize(text_data)

# Remove punctuation and make lowercase
tokens = [word.lower() for word in tokens if word.isalpha()]

# Count unique tokens before stemming
unique_tokens_before = set(tokens)

print(f"Number of unique tokens before stemming: {len(unique_tokens_be

# Apply Porter Stemmer
stemmer = PorterStemmer()

stemmed_tokens = [stemmer.stem(word) for word in tokens]

# Count unique tokens after stemming
unique_tokens_after = set(stemmed_tokens)

print(f"Number of unique tokens after stemming: {len(unique_tokens_aft
```

```
# Optional: show some examples
for original, stemmed in zip(tokens[:10], stemmed_tokens[:10]):
    print(f"{original} → {stemmed}")
```

```
[nltk_data] Downloading package punkt to
[nltk_data] C:\Users\Admin\AppData\Roaming\nltk_data...
[nltk_data] Package punkt is already up-to-date!
```

Number of unique tokens before stemming: 7121

Number of unique tokens after stemming: 5822

watch → watch

or → or

listen → listen

live → live

weekdays → weekday

at → at

mt → mt

at → at

subscribe → subscrib

via → via

Result: Number of unique tokens before stemming: 7121 Number of unique tokens after stemming: 5822 watch → watch or → or listen → listen live → live weekdays → weekday at → at mt → mt at → at subscribe → subscrib via → via

Question B.2.

- Perform lemmatization using NLTK lemmatizer. Count the number of

unique words/tokens before and after lemmatization

```
In [ ]: # Import the necessary libraries
import pandas as pd
import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

import unicodedata
import string
import re
```

```
In [ ]: # Import the corpus
corpus = pd.read_csv('./Data.csv')
```

```
In [ ]: # Check the first 5 documents
corpus.head()
```

Out []:

Data

- 0 Watch or listen live weekdays at 8:30am MT at ...
- 1 Watch or listen live weekdays at 8:30am MT at ...
- 2 Chubby And Hot, Always Stir The Pot!
- 3 Chubby And Hot, Always Stir The Pot!
- 4 Journalist, publisher of Rebel News — telling ...

```
In [ ]: # Function to clean corpus
def preprocess(word):
    # Remove URLs
    word = re.sub(r'http\S+|www\S+|https\S+', '', word, flags=re.MULTI
    # Remove '@' and hash '#' tags
    word = re.sub(r'@\w+', '', word)
    word = re.sub(r'#', '', word)
    word = re.sub(r'\brt\b', '', word, flags=re.IGNORECASE)
    # normalize unicode to ascii
    word = unicodedata.normalize('NFKD', word).encode('ascii', 'ignore
    # Remove standalone digits
    word = re.sub(r'\b\d+\b', '', word)
    # Casefold
    word = word.casefold()
    # Normalize whitespace
    word = ' '.join(word.split())
    # Remove some special characters while maintaining necessary ones
    word = re.sub(r"[^\w\s'-]", '', word)

    # handle slashes between words
    if '/' in word:
        # first split to maintain known combinations
        if word.lower() in {'a/b', 'w/d', 'w/o', 'b/c'}:
            return word.lower()
        # second split
        parts = word.split('/')
        if len(parts) == 2 and len(parts[0]) < 8 and len(parts[1]) < 8
            return '_'.join(parts) # mountain_river
        # third split for other cases
        return ' '.join(parts)

    return word
```

```
In [ ]: # Apply cleaning to corpus
corpus['clean_data'] = corpus['Data'].apply(preprocess)
```

```
In [ ]: # Tokenize
corpus['tokens'] = corpus['clean_data'].apply(word_tokenize)
```

```
In [ ]: # Total count of words before lemmatization
```

```
all_tokens = [token for lst in corpus['tokens'] for token in lst]
num_unique_before = len(set(all_tokens))

print(f'Number of unique words before lemmatization: {num_unique_before}')
```

Number of unique words before lemmatization: 7644

```
In [ ]: # Lemmatizer initialization
lemmatizer = WordNetLemmatizer()
```

```
In [ ]: # Lemmatization function
def lemmatize_text(word):
    tokens = word_tokenize(word)
    lemmatized_tokens = [lemmatizer.lemmatize(token) for token in tokens]
    return ' '.join(lemmatized_tokens)
```

```
In [ ]: # Count unique words after lemmatization
corpus['lemmas'] = corpus['clean_data'].apply(lemmatize_text)
all_lemmas = ' '.join(corpus['lemmas']).split()
num_unique_after = len(set(all_lemmas))

print(f'Number of unique words after lemmatization: {num_unique_after}')
```

Number of unique words after lemmatization: 7097

```
In [ ]: print(f'Reduction: {num_unique_before - num_unique_after} words')
```

Reduction: 547 words

Deep Seek Prompt

I need a function to properly handle slashes in my dataframe. I prefer a python solution to this problem

Question B.3.

Compare the change in word frequencies from stemming and lemmatization.

Which are the top 10 words after stemming/lemmatization?

```
In [1]: import pandas as pd
import re
import nltk
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.tokenize import word_tokenize
from collections import Counter
from nltk.corpus import stopwords
```

```
In [2]: # Create preprocess function
def preprocess_text(word):
```

```

"""Lowercase, remove punctuation and tokenize"""
word = word.lower()
# Remove characters that are not letters, apostrophes, white space
word = re.sub(r"[^a-zA-Z'\s-]", '', word)
tokens = word_tokenize(word)
return tokens

```

```

In [3]: # Process the data
def process_data(file_path, text_column, remove_duplicates=True):
    """Load CSV file, handle duplicates, and return tokens"""
    corpus = pd.read_csv(file_path)
    if remove_duplicates:
        corpus = corpus.drop_duplicates()
    word = ' '.join(corpus[text_column].astype(str))
    return preprocess_text(word)

```

```

In [4]: # Determine frequencies
def analyze_frequencies(tokens):
    """Compare stemmed vs lemmatized frequencies"""
    # Create stemmer and lemmer instances
    stemmer = PorterStemmer()
    lemmatizer = WordNetLemmatizer()

    # Original frequencies
    original_freq = Counter(tokens)

    # Stemmed frequencies
    stemmed_tokens = [stemmer.stem(token) for token in tokens]
    stemmed_freq = Counter(stemmed_tokens)

    # Lemmatized frequencies
    lemmatized_tokens = [lemmatizer.lemmatize(token) for token in tokens]
    lemmatized_freq = Counter(lemmatized_tokens)

    return original_freq, stemmed_freq, lemmatized_freq

```

```

In [5]: # Print comparisons

def print_comparison(original, stemmed, lemmatized):
    """Print top comparisons"""
    print("Unique Terms Count:")
    print(f"Original: {len(original)}")
    print(f"Stemmed: {len(stemmed)} (Difference: {len(original) - len(stemmed)})")
    print(f"Lemmatized: {len(lemmatized)} (Difference: {len(original) - len(lemmatized)})")

    print(f"\nTop 10 Original terms: {original.most_common(10)}")
    print(f"Top 10 Stemmed terms: {stemmed.most_common(10)}")
    print(f"Top 10 Lemmatized terms: {lemmatized.most_common(10)}")

```

```

In [7]: if __name__ == "__main__":

```

```

file_path = "/Users/obianujanuma/Library/CloudStorage/OneDrive-Pe
text_column = "Data"

tokens = process_data(file_path, text_column)
original, stemmed, lemmatized = analyze_frequencies(tokens)
print_comparison(original, stemmed, lemmatized)

```

Unique Terms Count:

Original: 7912

Stemmed: 6587 (Difference: 1325)

Lemmatized: 7368 (Difference: 544)

Top 10 Original terms: [('and', 857), ('the', 764), ('of', 671), ('to', 495), ('a', 448), ('in', 406), ('for', 359), ('i', 314), ('my', 299), ('is', 236)]

Top 10 Stemmed terms: [('and', 857), ('the', 764), ('of', 671), ('to', 495), ('a', 448), ('in', 407), ('for', 359), ('i', 314), ('my', 299), ('is', 236)]

Top 10 Lemmatized terms: [('and', 857), ('the', 764), ('of', 671), ('a', 501), ('to', 495), ('in', 407), ('for', 359), ('i', 314), ('my', 299), ('is', 236)]

Question B.4.

```

In [ ]: # Installing NLTK and downloading necessary resources
!pip install nltk
import nltk
nltk.download('punkt')
nltk.download('wordnet')
nltk.download('stopwords')

```

Requirement already satisfied: nltk in /usr/local/lib/python3.11/dist-packages (3.9.1)

Requirement already satisfied: click in /usr/local/lib/python3.11/dist-packages (from nltk) (8.2.0)

Requirement already satisfied: joblib in /usr/local/lib/python3.11/dist-packages (from nltk) (1.5.0)

Requirement already satisfied: regex<=2021.8.3 in /usr/local/lib/python3.11/dist-packages (from nltk) (2024.11.6)

Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from nltk) (4.67.1)

[nltk_data] Downloading package punkt to /root/nltk_data...

[nltk_data] Unzipping tokenizers/punkt.zip.

[nltk_data] Downloading package wordnet to /root/nltk_data...

[nltk_data] Downloading package stopwords to /root/nltk_data...

[nltk_data] Unzipping corpora/stopwords.zip.

Out[]: True

```

In [ ]: # Importing required libraries
import pandas as pd

```

```
import string
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
from collections import Counter
import re
```

```
In [ ]: # Load CSV file provided for assignment
from google.colab import files
uploaded = files.upload()
```

Choose Files no files selected

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving Data.csv to Data.csv

```
In [ ]: # Taking the uploaded file
file_name = list(uploaded.keys())[0]
corpus = pd.read_csv(file_name)
```

```
In [ ]: # Using Preprocessing tools
stop_words = set(stopwords.words('english'))
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()
```

```
In [ ]: # Clean and tokenize Function
def preprocess(text):
    text = text.lower()
    text = text.translate(str.maketrans('', '', string.punctuation))
    tokens = text.split()
    return tokens
```

```
In [ ]: # Removing stop words Function
def remove_stop_words(words):
    return [word for word in words if word not in stop_words]
```

```
In [ ]: # Stemming function
def stem_words(words):
    return [stemmer.stem(word) for word in words]
```

```
In [ ]: #Lemmatization Function
def lemmatize_words(words):
    return [lemmatizer.lemmatize(word) for word in words]
```

```
In [ ]: # Flatten helper Function
def flatten(list_of_lists):
    return [item for sublist in list_of_lists for item in sublist]
```

```
In [ ]: # Preprocessing pipeline
tokens = corpus['Data'].dropna().apply(preprocess)
```

```
In [ ]: # Normalization technique only (before stop word removal)
```

```
stemmed_only = tokens.apply(stem_words)
lemmatized_only = tokens.apply(lemmatize_words)
```

```
In [ ]: #Normalization technique after stop word removal
filtered = tokens.apply(remove_stop_words)
stemmed_filtered = filtered.apply(stem_words)
lemmatized_filtered = filtered.apply(lemmatize_words)
```

```
In [ ]: # Flatten all
flat_stemmed_only = flatten(stemmed_only)
flat_lemmatized_only = flatten(lemmatized_only)
flat_stemmed_filtered = flatten(stemmed_filtered)
flat_lemmatized_filtered = flatten(lemmatized_filtered)
```

```
In [ ]: # Counting word frequencies
freqs = {
    'Stemming only': Counter(flat_stemmed_only).most_common(20),
    'Lemmatization only': Counter(flat_lemmatized_only).most_common(20),
    'Stemming after stop word removal': Counter(flat_stemmed_filtered),
    'Lemmatization after stop word removal': Counter(flat_lemmatized_f
}
```

```
In [ ]: # Displaying result
import pprint
pprint.pprint(freqs)
```

```
{'Lemmatization after stop word removal': [('alberta', 1015),
                                           ('news', 598),
                                           ('outdoor', 549),
                                           ('community', 542),
                                           ('canada', 520),
                                           ('adventure', 516),
                                           ('writer', 386),
                                           ('park', 380),
                                           ('follow', 361),
                                           ('love', 359),
                                           ('photographer', 357),
                                           ('calgary', 345),
                                           ('world', 326),
                                           ('travel', 301),
                                           ('lover', 291),
                                           ('account', 283),
                                           ('abparksambassador', 263),
                                           ('people', 261),
                                           ('sheher', 260),
                                           ('u', 251)],

 'Lemmatization only': [('and', 3514),
                        ('the', 2841),
                        ('of', 2223),
                        ('to', 2179),
                        ('a', 1771),
                        ('in', 1539),
```

```

('for', 1507),
('alberta', 1015),
('is', 922),
('my', 892),
('on', 827),
('i', 816),
('at', 638),
('your', 600),
('news', 598),
('with', 587),
('you', 567),
('outdoor', 549),
('community', 542),
('are', 521)],
'Stemming after stop word removal': [('alberta', 1015),
                                      ('outdoor', 688),
                                      ('news', 598),
                                      ('commun', 595),
                                      ('adventur', 530),
                                      ('canada', 520),
                                      ('travel', 453),
                                      ('follow', 446),
                                      ('love', 408),
                                      ('writer', 386),
                                      ('park', 380),
                                      ('natur', 374),
                                      ('photograph', 371),
                                      ('calgari', 345),
                                      ('world', 326),
                                      ('work', 300),
                                      ('live', 299),
                                      ('lover', 291),
                                      ('account', 288),
                                      ('polit', 281)],
'Stemming only': [('and', 3514),
                  ('the', 2841),
                  ('of', 2223),
                  ('to', 2179),
                  ('a', 1554),
                  ('in', 1539),
                  ('for', 1507),
                  ('alberta', 1015),
                  ('is', 922),
                  ('my', 892),
                  ('on', 827),
                  ('i', 816),
                  ('outdoor', 688),
                  ('at', 638),
                  ('your', 612),
                  ('news', 598),
                  ('commun', 595),
                  ('with', 587),

```

```
('you', 567),  
('adventur', 530)]}
```

Findings of word frequencies for process of normalization after stop word removal

For normalization, I have taken two methods (i.e. Stemming and Lemmatization), and displayed the result for top 20 words with highest frequency.

When we perform normalization only, then we have a lot of stop words with higher no of frequency but when we do normalization after stop word removal, we find only normal words and the stop words like ("the", "and","you","to", etc) gets vanished. But when we check the counts of normal words for the normalization and normalization after stop word removal, there is no change of frequency of normal words.

References

ChatGPT [First Prompt: "Provide me an example for the python code for normalization of a document and normalization of document after stop word removal."]

ChatGPT [Last Prompt: "Promide me an expample of code for normalization with both lemmatization and stemming process."]